Abstract Classes vs Interfaces

10 min read Updated 28 Jul 2026 Topic 8 of 65

Bookmark Mark done

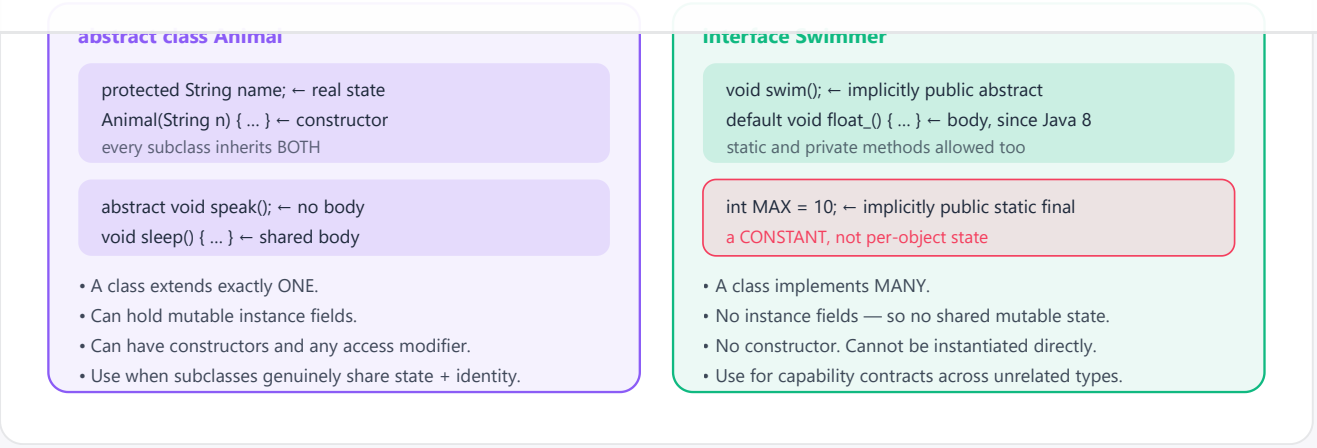
Why it matters

This is the most-asked design question in Java interviews, and since Java 8 gave interfaces `default` methods the old answer — "interfaces cannot have implementations" — is simply wrong. Candidates still recite it constantly, which makes an accurate answer a strong differentiator. The real distinction moved from *what they can contain* to *what they mean*: an abstract class models identity and shared state, an interface models capability.

The concept, simply

An **abstract class** is a partially built class. It can have constructors, mutable instance fields, any access modifier, and a mix of implemented and abstract methods. A class `extends` exactly one.

An **interface** is a contract. It can declare abstract methods, provide `default` implementations, and hold `static` and `private` helpers — but it has **no instance state** and no constructor. A class `implements` as many as it likes.



The line that still holds after Java 8: an abstract class can carry per-object state; an interface cannot. Its "fields" are implicitly public static final constants.

The decision rule that fits real code: ask whether the shared thing is **what the object is** or **what it can do**. A `Dog` *is an* `Animal` — one identity, shared state, abstract class. A `Dog` and a `Submarine` can both *swim* — a capability shared by otherwise unrelated types, interface.

How it works

	Abstract class	Interface
How many	extends **one**	implements **many**
Instance fields	Yes, any modifier, mutable	**No** — fields are implicitly `public static final`
Constructor	Yes (called via `super()`)	No
Method bodies	Yes	Yes — `default` and `static` since Java 8, `pr`
Method access	Any, including `protected`	Implicitly `public` (or `private` for helpers)
Instantiable	No	No

	Abstract class	Interface
Models	"is-a" — identity	"can-do" — capability

Why `default` methods were added is a genuinely good answer to give, because it explains a design constraint rather than a syntax rule. Java 8 needed to add `stream()`, `forEach()` and others to `Collection`. Adding an abstract method to an interface breaks every existing implementation — every `List` in every library on Earth would have failed to compile. `default` methods let an interface be extended with a working implementation, so old code keeps compiling. They exist for **backward-compatible interface evolution**, not to turn interfaces into abstract classes.

Multiple interfaces with the same default method create an ambiguity, and Java refuses to guess:

JAVA Copy

```java
interface A { default String hello() { return "A"; } }
interface B { default String hello() { return "B"; } }

// class C implements A, B { }
// ✗ "class C inherits unrelated defaults for hello() from types A and B"

class C implements A, B {
    @Override public String hello() {
        return A.super.hello();     // explicit, unambiguous resolution
    }
}
```

That `A.super.hello()` syntax is the diamond problem being solved by *forcing a decision* — which is exactly why default methods did not reintroduce the problem multiple class inheritance would have. And when a class inherits both a superclass method and an interface default with the same signature, **the class wins**: real implementations beat defaults.

Code examples

The classic combination — an abstract class for identity, interfaces for capability:

JAVA

Copy

```
abstract class Animal {
    protected final String name;                    // shared STATE
    protected Animal(String name) { this.name = name; }

    abstract String speak();                        // every animal must answ
    void sleep() { System.out.println(name + " sleeps"); }   // shared beh
}

interface Swimmer { void swim(); }
interface Trainable {
    void learn(String trick);
    default void praise() { System.out.println("Good!"); }   // opt-in def
}

// Dog IS an Animal, and CAN swim and be trained.
class Dog extends Animal implements Swimmer, Trainable {
    Dog(String name) { super(name); }
    String speak() { return "Woof"; }
    public void swim() { System.out.println(name + " paddles"); }
    public void learn(String trick) { System.out.println(name + " learns "
}

// Submarine is not an Animal at all — but it can swim.
class Submarine implements Swimmer {
    public void swim() { System.out.println("Dive!"); }
}

// And now one method serves both, with no common ancestor:
static void race(List<Swimmer> competitors) { competitors.forEach(Swimmer:
```

That last line is the payoff. `Dog` and `Submarine` share no class hierarchy, and an abstract class could never have united them.

What an interface cannot do — and the trap it creates:

JAVA

Copy

```
    int MAX = 100;              // implicitly public static final — a CONSTANT
    // int current;             // ✗ will not compile: no instance fields

    void increment();
    default void reset() { /* cannot touch per-object state — there is non
}
// This is the real limit of default methods: they can only call other
// interface methods. They cannot read or write per-instance state, which
// is exactly why they did not make abstract classes redundant.
```

Functional interfaces, the bridge to Chapter 7:

JAVA Copy

```
@FunctionalInterface                // compiler enforces exactly one abstra
interface Validator {
    boolean test(String input);     // the single abstract method
    default Validator and(Validator other) {     // defaults do not break
        return s -> this.test(s) && other.test(s);
    }
}

Validator notBlank = s -> !s.isBlank();
Validator shortEnough = s -> s.length() <= 20;
Validator both = notBlank.and(shortEnough);
// A lambda can implement an interface with one abstract method. It can ne
// implement an abstract class — another reason the two are not interchang
```

Under the hood

Interface calls compile to `invokeinterface` rather than `invokevirtual`, and historically that was slower: a class's position in a vtable is fixed by its hierarchy, but a class can implement many interfaces, so the JVM cannot use a simple fixed offset and must search or consult a cache. In modern HotSpot the gap is negligible for monomorphic call sites, which the JIT devirtualises and inlines exactly as it does virtual calls. It is not a reason to prefer abstract classes.

it, which gives you an interface's flexibility *and* a closed set the compiler can reason about — so a `switch` over its implementations can be checked for exhaustiveness. For modelling a fixed set of alternatives (a payment result that is `Approved`, `Declined` or `Pending`), a sealed interface plus records is now often better than either classic option. Chapter 10 covers it properly.

Common mistakes

- **Saying "interfaces cannot have method bodies".** Outdated since Java 8. The accurate line is that interfaces cannot have *instance state*.
- **Treating a field in an interface as per-object state.** It is `public static final` — a shared constant.
- **Using an abstract class purely to share a helper method.** That burns the single inheritance slot. A `static` utility or composition is usually better.
- **Using default methods as a dumping ground.** They exist for compatible evolution. Piling logic into them creates behaviour with nowhere to keep state and no clean way to test.
- **Forgetting `public` when implementing an interface method.** Interface methods are implicitly public and access cannot narrow, so omitting it will not compile.
- **Interface-per-class by reflex.** An interface with exactly one implementation and no test seam is indirection without benefit.

Interview lens

> **Q: "Abstract class vs interface — when do you use each?"**
> A: An abstract class when the implementations genuinely share identity and state — it can hold instance fields and a constructor, and a class gets only one. An interface when I am describing a capability that unrelated types can offer, or when I need a type to have several of them. Since Java 8 both can carry method bodies, so the deciding line is state: interfaces have none.

A: To evolve interfaces without breaking implementations. Java 8 needed to add `stream()` and `forEach()` to `Collection`; as abstract methods, that would have broken every existing implementation on the planet. A `default` gives a working body so old code keeps compiling. Backward-compatible evolution, not a redesign of interfaces.

Q: "Do default methods bring back the diamond problem?"

A: No, because there is still no inherited *state* — that was the real problem with multiple class inheritance. If two interfaces supply the same default signature, the class simply will not compile until you override it, and you can delegate explicitly with `A.super.method()`. And a concrete method inherited from a superclass always beats an interface default.

Q: "Can an interface have a constructor or instance variables?"

A: Neither. No constructor, because there is no instance state to initialise and interfaces are never instantiated directly. Any field you declare is implicitly `public static final` — a constant shared by everyone, not per-object state.

Q: "Can an abstract class have no abstract methods?"

A: Yes. `abstract` on the class only means "cannot be instantiated". A class with all methods implemented can still be abstract, which is a reasonable way to say "this is a base to extend, never to use directly".

Q: "What is a functional interface?"

A: One with exactly one abstract method, so a lambda or method reference can implement it. Default and static methods do not count toward that one. `@FunctionalInterface` makes the compiler enforce it. Note that this is something an abstract class can never be — lambdas only target interfaces.

A: Often, yes. If I am modelling a closed set of alternatives, a `sealed interface` with record implementations gives me interface flexibility plus a set the compiler knows is complete — so a switch over it is checked for exhaustiveness and adding a case breaks the build in the right place. That is now my default for sum types, with abstract classes reserved for genuine shared state.

Cheat-card

- The line since Java 8 is **state**, not method bodies: interfaces have **no instance fields**.
- Abstract class = **is-a**, one only, constructor + mutable fields. Interface = **can-do**, many.
- Interface fields are `public static final` constants. Methods implicitly `public`.
- `default` exists for **backward-compatible evolution** (Java 8 added `stream()` to `Collection`).
- `static` in interfaces (8), `private` helpers (9).
- Conflicting defaults → compile error → resolve with `A.super.m()`. **Class beats interface.**
- Lambdas target **functional interfaces only** — never abstract classes.
- Closed set of alternatives? Reach for a **sealed interface + records** (Java 17).

QUICK CHECK — no marks, just for you.

1. Since Java 8 gave interfaces default methods, what is the remaining fundamental difference from an abstract class?

- ○ Abstract classes can be instantiated directly
- ○ Interfaces cannot be used as types
- ○ Interfaces have no instance state — an abstract class can hold mutable per-object fields and a constructor

Interfaces still cannot contain any method bodies

Check answer

2. Why were default methods added in Java 8?

- ◯ To replace abstract classes entirely
- ◯ To allow interfaces to hold mutable state
- ◯ To make lambdas possible
- ◯ To let interfaces be evolved without breaking existing implementations — adding stream() to Collection as an abstract method would have broken every List in existence

Check answer

3. A class implements two interfaces that both declare the same default method. What happens?

- ◯ It will not compile until the class overrides the method — it can delegate explicitly with A.super.method()
- ◯ Both run, in declaration order
- ◯ The JVM picks one at random at run time
- ◯ The first interface in the implements clause wins

Check answer

LEARN MORE

ARTICLES & DOCS

- Interfaces (JLS §9)
  docs.oracle.com
- Default methods — The Java Tutorials
  docs.oracle.com
- Sealed classes (JEP 409)
  openjdk.org

VIDEOS

- "abstract class vs interface" — video explainers
  YouTube
- abstract class vs interface interview questions
  YouTube

Hand-picked references for this lesson.